

Assignment 9.3

2303A51144

K.Sanskar

Batch 17

Task 1: Basic Docstring Generation

Scenario

You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.

Requirements

- Write a Python function to return the sum of even numbers and sum of odd numbers in a given list
- Manually add a Google Style docstring to the function
- Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
- Compare the AI-generated docstring with the manually written docstring
- Analyze clarity, correctness, and completeness

Expected Output

- Python function with manual Google-style docstring
- AI-generated docstring for the same function
- Comparison explaining differences between manual and AI-generated documentation
- Improved understanding of AI-generated function-level documentation

Prompt

#generate a docstring generation code and generate code for developing a utility function that processes numerical lists and must be properly documented for future maintenance

#generate for sum of odd numbers in the list and sum of even numbers in the list

Code

```
def process_numerical_list(num_list):
```

```
    if not isinstance(num_list, list):
```

```

    raise ValueError("Input must be a list.")

if not all(isinstance(num, (int, float)) for num in num_list):
    raise ValueError("All elements in the list must be numeric.")
#generate for sum of odd numbers in the list and sum of even numbers in the list

odd_sum = sum(num for num in num_list if num % 2 != 0)
even_sum = sum(num for num in num_list if num % 2 == 0)
return odd_sum, even_sum

#take user input for the list of numbers
while True:
    user_input = input("Enter a list of numbers separated by commas: ")
    try:
        num_list = [float(num.strip()) for num in user_input.split(",")]
        odd_sum, even_sum = process_numerical_list(num_list)
        print(f"Sum of odd numbers: {odd_sum}")
        print(f"Sum of even numbers: {even_sum}")
        break
    except ValueError as e:
        print(f"Invalid input: {e}. Please try again.")

```

Output

```

Enter a list of numbers separated by commas: 1,2,3,4,5,6,7
Sum of odd numbers: 16.0
Sum of even numbers: 12.0

```

Task 2: Automatic Inline Comments

Scenario

You are developing a student management module that must be easy to understand for new developers.

Requirements

- Write a Python program for an `sru_student` class with the following:
 - Attributes: `name`, `roll_no`, `hostel_status`
 - Methods: `fee_update()` and `display_details()`
- Manually write inline comments for each line or logical block
- Use an AI-assisted tool to automatically add inline comments
- Compare manual comments with AI-generated comments
- Identify missing, redundant, or incorrect AI comments

Expected Output

- Python class with manually written inline comments
- AI-generated inline comments added to the same code
- Comparative analysis of manual vs AI comments
- Critical discussion on strengths and limitations of AI-generated comments

Prompt

```
#create a class sru_student with attributes name, roll_no, hostel_status
#take input from user and Methods: fee_update() and display_details()
```

Code

```
#create a class sru_student with attributes name, roll_no, hostel_status
class sru_student :
    def __init__(self, name, roll_no, hostel_status):
        self.name = name
        self.roll_no = roll_no
        self.hostel_status = hostel_status
    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Hostel Status:", self.hostel_status)
#take input from user and Methods: fee_update() and display_details()
    def fee_update(self, amount):
```

```
print("Fee updated by", amount)
name = input("Enter name: ")
roll_no = input("Enter roll number: ")
hostel_status = input("Enter hostel status: ")
student = sru_student(name, roll_no, hostel_status)

amount = float(input("Enter fee amount to update: "))

student.display()
student.fee_update(amount)
```

Output

```
Enter name: vishnu
Enter roll number: 101
Enter hostel status: hostel
Enter fee amount to update: 300
Name: vishnu
Roll No: 101
Hostel Status: hostel
Fee updated by 300.0
```

Task 3: Module-Level and Function-Level Documentation

Scenario

You are building a small calculator module that will be shared across multiple projects and requires structured documentation.

Requirements

- Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)
- Manually write NumPy Style docstrings for each function
- Use AI assistance to generate:
 - A module-level docstring
 - Individual function-level docstrings

- Compare AI-generated docstrings with manually written ones
- Evaluate documentation structure, accuracy, and readability

Expected Output

- Python script with manual NumPy-style docstrings
- AI-generated module-level and function-level documentation
- Comparison between AI-generated and manual documentation
- Clear understanding of structured documentation for multi-function scripts

Additional Requirement

- Push the complete project documentation as a .md file to a GitHub repository
- Ensure documentation covers module overview and function descriptions

Prompt

#generate a python code for a small calculator module that will be shared across multiple projects and requires structured documentation.

#add, subtract, multiply, divide

#module-level docstring

Code

```
def add(a, b):

    return a + b

def subtract(a, b):

    return a - b

def multiply(a, b):

    return a * b

def divide(a, b):

    if b == 0:

        raise ValueError("Cannot divide by zero.")

    return a / b
```

```
# Example usage:

if __name__ == "__main__":
    print("Addition:", add(5, 3))
    print("Subtraction:", subtract(5, 3))
    print("Multiplication:", multiply(5, 3))
    print("Division:", divide(5, 3))

    try:
        print("Division by zero:", divide(5, 0))
    except ValueError as e:
        print("Error:", e)
```

Output

```
Multiplication: 15
Division: 1.6666666666666667
Error: Cannot divide by zero.
```